PA#1 — Proposal and planning

Oct 17, 2026 · @Thư

Team: 22127403 — Nguyễn Trần Minh Thư; 22127253 — Hoàng Trí Luật. Repository: *[paste GitHub link]*

1. Product: who it is for and the problem

User: Mr. Hùng, head cook of a 40-table broken-rice (cơm tấm) restaurant using QR table-ordering (table-order). In the 11:00–13:00 rush he gets about 120 orders; some guests type notes like "no onions, my child is allergic to peanuts". *(Assumed persona.)*

Problem: free-text allergy notes are easy to miss or misread when the kitchen is overloaded. Today he reads them by eye on the printed ticket.

2. The LLM feature and the cost of being wrong

Feature (one): turns a guest's free-text note into structured flags (`allergen`, `remove ingredient`, `spice level`) with a confidence. If the note is ambiguous (e.g. "slightly allergic to seafood"), the guest gets one confirmation question on their phone before the order reaches the kitchen. Flags show prominently on the kitchen screen.

If it is wrong, who pays, how much, can it be undone:

- **Missed allergen (worst case):** the guest eats a dish containing an allergen. Direct health harm that cannot be undone; the restaurant may be liable.
- **Extra flag or extra question:** the kitchen and the guest lose a few seconds. Can be undone.

How we would know: about 100 hand-labelled Vietnamese notes; we measure recall of the allergen flag (more important than precision).

Where the gate sits: only on ambiguous notes (the guest confirms), plus the kitchen as the final check. Not on every note (too slow). When unsure, ask or flag; never ignore a note.

3. Scope this semester

In scope: extracting flags from notes; asking the guest when a note is ambiguous; showing flags on the kitchen screen.

Out of scope: nutrition or medical advice; dish suggestions; reading ingredients from the menu; payment and bill splitting (IA#1).

4. Plan and ownership

Checkpoints are PA#1–PA#6. Only the PA#1 date is confirmed; the rest are estimates.

Checkpoint	Date	Thư	Luật
PA#1 Proposal	22/10	Parts 1–3	Parts 4–6, self-assessment, repo, CI
PA#2 CI	~19/11	Convention file, lint	Tests, CI gate, labelled notes
PA#3 Prototype	~3/12	Extraction, ask-back API	Guest UI, kitchen screen
PA#4 Eval	~10/12	Guardrails, human gate	Eval suite, threshold
PA#5 Final	~17/12	LLM logs, metrics	Incident, post-mortem
PA#6 Oral	TBD	Cross-review so both can explain all code	

5. Risks

1. **Too few labelled notes to evaluate** (blocks PA#4 and IA#4). This week each of us writes 25 notes; friends add more to reach 100; we cross-label.
2. **The model misses unusual wording** (a missed allergen is the worst failure). Hard cases go into the test set early; low confidence always triggers ask-the-guest.

6. Technology choices

- **Model:** Claude Haiku 5.5 via API (planned): small, fast, enough for short notes.
- **Platform and CI:** our table-order system *[fill in stack]*; GitHub Actions blocks merges on red tests, and the recall eval runs there from PA#4.
- **Data sent outside:** note text only; no name, phone number or table code.
- **Cost:** 200 notes/day ≈ 60,000 in + 16,000 out tokens ≈ $0.014/day at $0.10 / $0.50 per million.